

TP01- SERIES CHRONOLOGIQUES : PHASE EXPLORATOIRE

Objectif : série chronologique : phase exploratoire

1. VISUALISATION DE SERIES

Chronogramme : graphe de $t \rightarrow x_t$

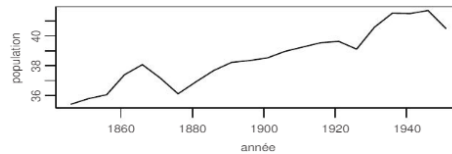


Figure 1. Population de la France en millions

Lag plot ou courbes de décalage : diagramme de dispersion des points (x_{t-k}, x_t) . Ce graphique permet de vérifier :

- Les valeurs aberrantes
- La saisonnalité ou au contraire un caractère aléatoire.
- Une corrélation sérielle. Si le diagramme retardé suggère une corrélation entre les deux séries, on dit que la série présente une autocorrélation d'ordre k.

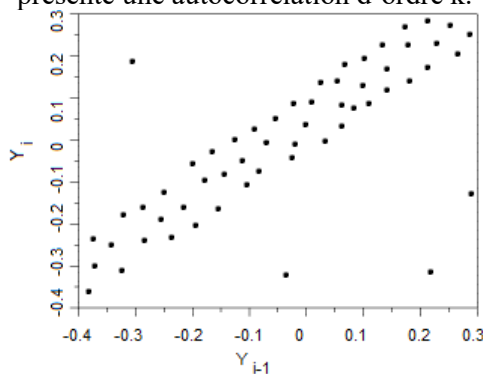


Figure 2. Lag plot pour $k=1$, détection outliers

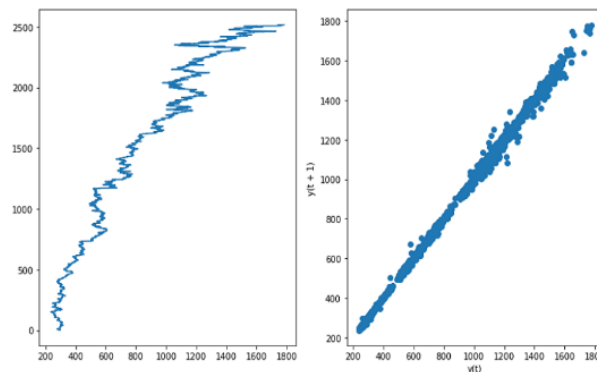


Figure 3. Données boursières : chronogramme et lag-plot($k=1$)

Monthplot of average Irish Rainfall 1870-1879

Month plot : représentation simultanée des chronogrammes des séries associées à chaque saison ou mois. Les points du chronogramme du mois m correspondent à la série du mois m , pour toutes les années. C'est une représentation pertinente pour étudier la saisonnalité d'une série.

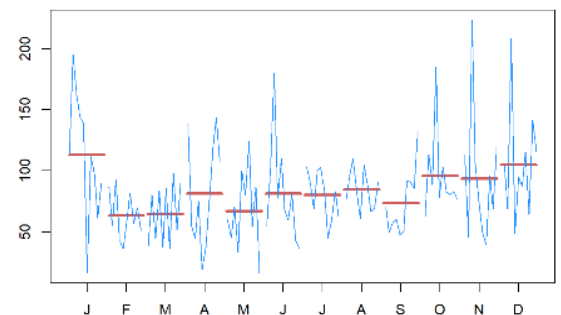


Figure 4. Month plot : pluie en Irlande (1870-1879)

2. AIDE SOUS R

Quelques packages :

Package	Description
fpp3	Librairies qui fournissent des objets chronologiques qui peuvent gérer des dates et des heures.
datasets	
Zoo	
tseries	
Xts	Prévision des séries temporelles
Chron	
forecast	
smooth	
prophet	

Installation et appel d'un package

```
install.packages("xts"); library(xts)
```

```
# data analysis with xts format
```

L'objet *ts*

Une série temporelle peut être considérée comme $\left\{ \begin{array}{l} \text{une liste de nombres} \\ \text{les moments où ces nombres ont été enregistrés} \end{array} \right.$. Ces informations peuvent être stockées sous la forme d'un objet *ts* dans R.

```
serie = co2
is.ts(serie)
vecteur_en_ts=ts(un_vecteur,start=c(1982,1),frequency=12) #vecteur→serie
?(serie)           #descripteurs/ information sur objet « time-séries »
time(serie)         #les dates de la série
frequency(serie)    #la fréquence de la série
```

La fréquence est le nombre d'observations avant que la tendance saisonnière ne se répète.³ Lors de l'utilisation de la fonction *ts()* dans R, les choix ci-contre doivent être effectués.

Annuelle	1	Mensuelle	12
Trimestrielle	4	Hebdomadaire	52

```
tsp(serie)           #Obtenir le début, la fin et la fréquence de la série
serie_sous_serie=window(serie,c(1970,4),1980) #Extraire une sous-série
matrix(serie,12,10)  #Représentations annuelles de la série
matrix(serie,10,12)  #Comparez
ts.plot(matrix(serie,12,10)) #Graphes
ts.plot(matrix(serie,10,12)) #Comparez
```

Tracer des séries

```
plot.ts(serie_1,)
ou
seqplot.ts(serie_1, serie_2, colx = "black", coly = "red",)
legend("topleft", c("serie1", " serie2"),col=1:2,lty=1:1)
```

Remarque : Si on applique la fonction *monthplot()* à un vecteur numérique, on indique la saisonnalité par l'option *frequency=*

Décomposition de séries

```
#decompose() décompose une ts avec fréquence en 3 composantes : tend., sais., résidus
serie_decomp = decompose(serie,type = c("additive"), filter = NULL)
plot(serie_decomp)           # serie_decomp$figure
```

Estimation de la tendance

```
y= serie_decomp$trend # Estimation non-paramétrique/paramétrique de la tendance
t = time(serie)
t_carre = time(serie)**2
modele_lin= lm(mt_hat_1~t)  #Estimation paramétrique de la tendance = a + bt
modele_quad= lm(mt_hat_1~t+t_carre) #Estimation paramétrique de la tendance = a + bt + ct^2.
modele_lin$coefficients    #Valeurs des coefficients a, b et c
summary(modele_lin)

plot(serie_decomp$trend)
abline(modele_lin,col= « red »)

mt_hat_3=ts(modele_quad$fitted,start=start(serie), frequency = frequency((serie)))
seqplot.ts(mt_hat_1, mt_hat_3,)
```

1. Exercice. Taches solaires.

1. Charger les données *sunspots*.

```
Data=sunspots
```

Décrire les données

Sujet :

Type de données :

Nombre d'entrées (nombre d'entrées) :

Pas de temps :

Début :

Fin :

```
?sunspots
is.ts(Data)
time(Data)
start(Data)
frequency(Data)
```

2. Plotter la série (avec titre et libellés des axes).

3. Observez-vous

une tendance ?

une saisonnalité ?

4. Décomposer la série par un modèle additif (utiliser la fonction *decompose()*) et dresser le graphique de la décomposition de la série

5. On veut établir la droite de régression linéaire qui approxime les valeurs de la tendance.

Etablir la régression linéaire (fonction *lm()*) de la tendance par le temps (ici obtenue par la variable *time(ResultatDeLaDecomposition\$trend)*).

```
# a compléter
y =
x =
reglin= lm()

#Tracer la courbe
plot.ts(y)
abline(reglin,col="red")
```

6. Donner les coefficients de la droite de régression :

En vous appuyant sur les coefficients de la droite de régression, donner une estimation du hausse des taches de soleil sur le temps de l'étude (faire un produit en croix liant temps et hausse) :

7. Décomposer la série par un modèle multiplicatif (utiliser la fonction *decompose()*) et dresser le graphique de la décomposition de la série.

2. Exercice. Volume de fret pour les aéroports de Paris.

On considère le fichier *fret.txt* qui contient le volume de fret, exprimé en millions de tonnes, pour les aéroports de Paris entre janvier 1982 et décembre 2005 (source INSEE).

1. Charger les données. Réaliser la description de chacune des variables présentes dans ce fichier (type et codage, colonnes, début, fin, pas de temps).

```
library(readr)
```

```
data=read.table("./Data/fret.txt",sep=";")
View(data)
data[1:10,]
```

2. On souhaite sélectionner les données de fret (colonne 4, commande `Serie=data[,4]`), et les placer dans une variable nommée `série`. Comme les données originales vont de la valeur la plus récente à la valeur la plus ancienne, il faut « inverser » la série (commande `rev()`), vous stockerez cette série ordonnée dans une variable `SerieOrd`.

Après inversion, vérifier que la 100^{ème} valeur de votre série est 79.2

3. Pour visualiser une chronique, il convient dans un premier temps de transformer l'objet série de type `vector` en un objet de type `ts` (pour *time séries*). Ecrire et expliquer (les options) la ligne de code suivante :

```
série.ts=ts(série,start=c(1982,1),frequency=12)
```

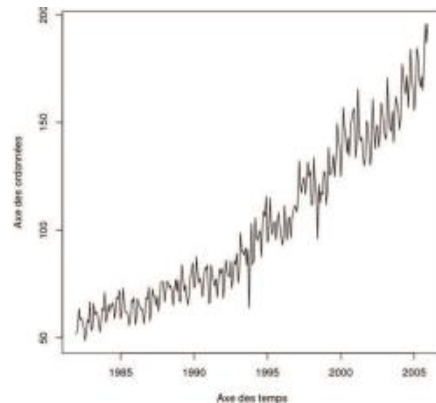
4. Comparer le contenu des objets `sérieOrd` et `série.ts` (regarder dans environnement).

Quelle est la différence ?

5. Donner les indicateurs de statistique univariées classiques (moyenne, médiane, 1^{er} et 3^{ème} quartiles, écart-type, fonction `summary()` et `sd()`).

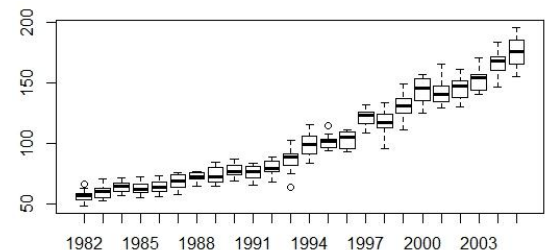
Quelle est la moyenne mensuelle du fret (en millions de tonnes) sur l'ensemble des années d'étude ?

6. Représenter la série comme ci-contre (fonction `plot.ts()`). Ajouter un titre.



7. On souhaite pour chaque année obtenir la représentation d'un boxplot. Afin de faciliter les comparaisons, on impose que les boxplots soient représentées sur le même graphique en parallèle. Ajouter titre et légende des axes.

```
boxplot(data[,4]~+data[,3])
```

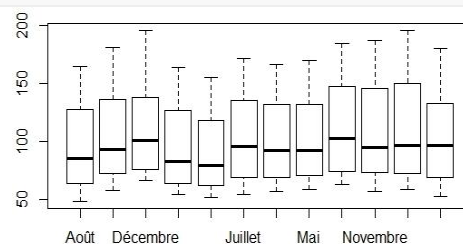


8. On veut également obtenir des informations quantitatives (moyennes, médianes, quartiles et extrêmes) concernant chaque année d'étude. Ecrire et expliquer les lignes de code suivantes :

```
for (i in 1982:2005){
  cat("année :",i,"\n")
  print(summary(data[data[,3]==i,4]))
}
```

9. On envisage d'effectuer un travail similaire pour les mois. Réaliser le boxplot de toutes les valeurs pour chaque mois (figure ci-contre). Ajouter titre et libellés des axes.

Comment pourrait-on améliorer cette figure ?



10. On veut compléter cette représentation graphique avec des informations quantitatives. S'inspirer de la boucle écrite en 8, pour générer une boucle donnant les principaux indicateurs statistiques pour chaque mois.

3. Exercice. Ventes.

1. Charger les données de vente BjSales et les expliquer (quoi ? début ? fin ? fréquence ?)

```
?BJsales
```

2. Plotter la série (avec titre et libellés des axes).

3. Observez-vous

une tendance ?

une saisonnalité ?

4. Décomposer la série par un modèle additif (utiliser la fonction *decompose()*) et dresser le graphique de la décomposition de la série

5. Pourquoi le logiciel n'arrive pas à décomposer la série chronologique ?

TP02- SERIES CHRONOLOGIQUES : MOYENNES MOBILES, DECOMPOSITION, MODELISATION, PREDICTION

Objectif : série chronologique : moyennes mobiles, décompositions, modélisation, prédiction

1. AIDE SOUS R pour les séries chronologiques

Décomposition de séries

```
#decompose() décompose une ts avec fréquence en 3 composantes : tendance, saisonnier, résidus
serie_decomp = decompose(serie,type = c("additive"), filter = NULL)
plot(serie_decomp)          # serie_decomp$figure
```

Estimation de la tendance

```
mt_hat_1= serie_decomp$trend # Estimation non-paramétrique paramétrique de la tendance
t = time(serie)
t_carre = time(serie)**2
mt_hat_2= lm(mt_hat_1~t+t_carre) #Estimation paramétrique de la tendance = a + bt + ct^2.
mt_hat_2$coefficients          #Valeurs des coefficients a, b et c
mt_hat_3=ts(mt_hat_2$fitted,start=start(serie), frequency = frequency((serie)))
seqplot.ts(mt_hat_1, mt_hat_3,)
```

Etude des résidus

```
re=m$random          #appel des résidus généré par la fonction decompose
plot(m$random) #plot des résidus
res=CVS-CVS.lm$fitted.values #Définition des résidus
res=res/sqrt(var(res)) #Définition des résidus réduits
acf(res) #Corrélogramme des résidus
hist(res) #Histogramme des résidus
qqnorm(res) #Comparaison des quantiles des résidus
abline(0,1) # et des quantiles d'une loi normale
no=stl(co2, "per")
plot(no)
nx=no$time.s\series[,"remainder"]
acf(nx,na.action=na.pass)
```

1. Exercice 1. Volume de fret pour les aéroports de Paris.

On considère le fichier `fret.txt` qui contient le volume de fret, exprimé en millions de tonnes, pour les aéroports de Paris entre janvier 1982 et décembre 2005 (source INSEE).

A - Phase exploratoire

1. Charger le code ci-dessous. A la fin de chaque bloc, vous expliquerez ce que le bloc de code réalise.

```
-----Bloc 1-----
library(readr)
data=read.table("VotreChemin/Data/fret.txt", sep=";")
View(data)
data[1:10,]
-----Bloc 2-----
série=data[,4]
série=rev(série)
série[100]
-----Bloc 3-----
série.ts=ts(série, start=c(1982,1), frequency=12)
-----Bloc 4-----
#série est simplement un vecteur
#série.ts est un objet spécifique de R « time-séries object » il s'agit d'une série
#chronologique indexée sur le temps
-----Bloc 5-----
sd(série.ts)
summary(série.ts)
-----Bloc 6-----
plot.ts(série.ts)
-----Bloc 7-----
boxplot(data[,4]~ data[,3])
-----Bloc 8-----
for (i in 1982:2005){
  cat("année :",i,"\n")
  print(summary(data[data[,3]==i,4]))
}
-----Bloc 9-----
boxplot(data[,4]~data[,2])
-----Bloc 10-----
for (k in unique(data[,2])){
  cat("Mois :",k,"\n")
  print(summary( data[data[,2]==k,4]))
}
```

B - Phase de modélisation : modèle affine

1. On veut modéliser la série à l'aide d'un modèle affine de la forme $y_t = at + b$. Pour cela, on définit l'objet `t` qui contiendra les entiers entre 1 et 288 correspondant aux 288 mois d'étude (nombre de mois que l'on peut obtenir). Ecrire et expliquer les lignes de code suivantes :

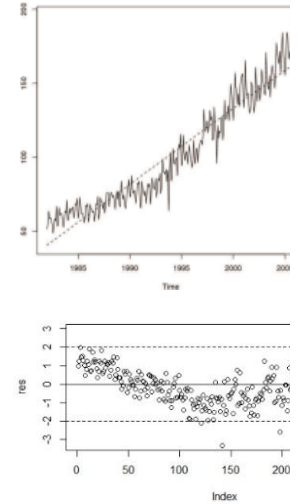
```
length(série)      #Explication
t=1:288            #Explication
modell=lm(série~t) #Explication (?lm)
```

Remarque : l'objet *modell* contient un certain nombre d'informations sur la modélisation. Pour obtenir l'ensemble de ces informations, regroupées dans différents objets, il suffit d'écrire *summary(modell)*

2. Donner une estimation des paramètres a et b du modèle *modell*. Commenter la valeur de a .

3. On souhaite voir sur un graphique la série initiale et la modélisation de la composante tendancielle. Ecrire le code suivant :

```
tendance1.ts=ts(modell1$fitted.values,start=c(1982,1),frequency=12)
ts.plot(série.ts,tendance1.ts, # on trace 2 series
       lty=c(1,2),           # chaque serie a un type de ligne
       col= "black", "red")   # chaque serie a une couleur
```



4. Représenter le graphique des résidus normalisés ci-contre (fonction *plot()* et *abline()*). Les résidus normalisés sont générés de la façon suivante :

```
#génération des résidus normalisés
n=length(série)
s=sqrt((n-1)*var(modell1$residuals)/n )
res=modell1$residuals/s
```

C - Phase de modélisation : modèle exponentiel

1. On veut maintenant modéliser la série à l'aide d'un modèle exponentiel de la forme $y_t = \exp(at + b)$. Quel changement de variable doit-on réaliser afin de se ramener à un modèle affine ?

2. Calculer le logarithme de la série en écrivant le code suivant :

```
log.série=log(série)
```

3. Via la fonction *lm* écrire une ligne de code permettant de réaliser l'ajustement affine de *log.série* par rapport au temps *t*. On appellera ce nouveau modèle *model2*. Donner une estimation des paramètres *a* et *b*.

4. Comment obtient-on les valeurs estimées $\hat{y}_t$ associées au modèle (théoriquement pas en ligne de commande) ?

5. Calculer l'exponentielle des valeurs estimées, puis représenter sur un même graphique la série initiale et la courbe de tendance obtenue via le modèle exponentiel. Ajouter le titre et les libelles de axes au graphique

```
tendance2=exp(model2$fitted.values)
tendance2.ts=ts(tendance2,start=c(1982,1),frequency=12)
ts.plot(série.ts,tendance2.ts,lty=c(1,2))
```

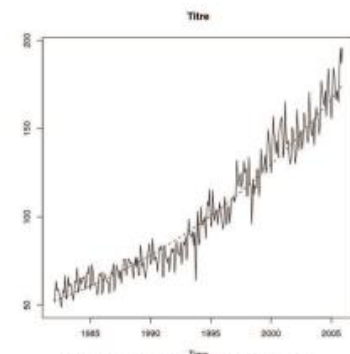
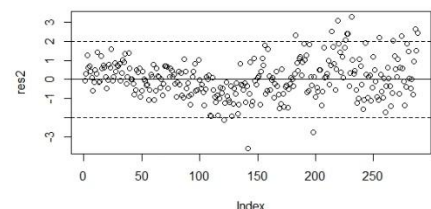


Fig. 7 : Modèle exponentiel

6. Comparer les représentations graphiques des modèles affine et exponentiel.

7. En vous inspirant de la question B4, sans aucune transformation (sauf une normalisation judicieuse) étudier les résidus associés à cette seconde modélisation (*residus2=série-tendance2*).



D - Modèle quadratique

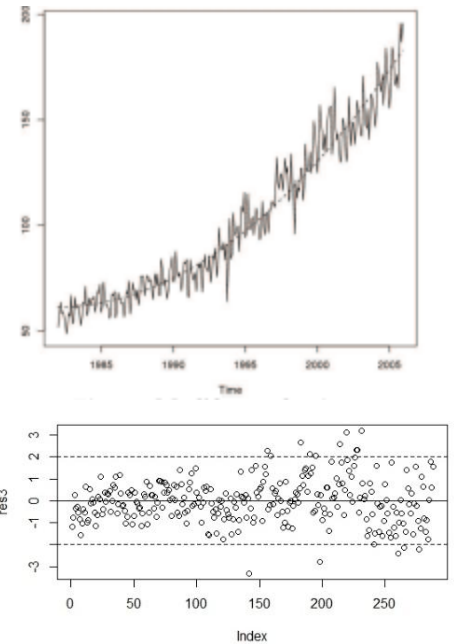
On se propose de modéliser la série via un modèle quadratique : $y_t = at^2 + b$.

1. La fonction `lm()` écrire une ligne de code permettant de réaliser l'ajustement quadratique de séries par rapport au temps t . On appellera ce nouveau modèle `model3` (faire usage de la notation t^2 pour symboliser t^2).

2. Donner une estimation des paramètres a et b

3. Visualiser sur un même graphique la série initiale et l'ajustement de sa composante tendancielle via le modèle quadratique. Ajouter le titre et les libelles de axes au graphique.

4. En vous inspirant de la question B4, étudier les résidus associés à cette seconde modélisation (`residus3=série-tendance3`). Ajouter le titre et les libelles de axes au graphique.

E - Sélection du meilleur modèle

1. D'après la phase précédente, on note que trois modélisations ont été envisagées pour ajuster la composante tendancielle de la série. Il convient dès lors de choisir sur la base d'un critère adéquat le meilleur modèle. Par rapport au critère visuel, quel modèle semble le meilleur ?

2. L'un des critères souvent utilisé est celui de la somme des résidus carrés. Pour chacun des modèles, déterminer cette somme des résidus carrés (faire usage de la fonction `sum()`).

3. Comparer les valeurs et dire quel modèle est le meilleur selon ce critère.

2. Exercice. Décomposition manuelle de séries : modèle additif.

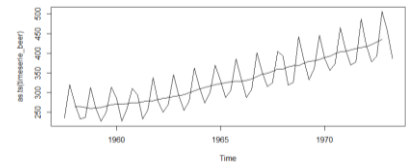
La production de bière est un bon exemple de série chronologique additive. Lorsque les valeurs métriques augmentent, la saisonnalité reste relativement constante.

1. Charger les données et les tracer.

```
install.packages("fpp")
install.packages("forecast")
library(fpp)
library(forecast)
data(ausbeer)
timeserie_beer = tail(head(ausbeer, 17*4+2), 17*4-4)
plot(as.ts(timeserie_beer))
```

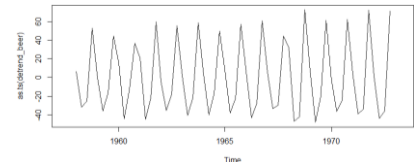
2. Détection de la tendance.

La production australienne de bière suit clairement la saisonnalité annuelle. Comme elle est enregistrée trimestriellement, il y a 4 points de données enregistrés par an, et nous utilisons une fenêtre moyenne mobile de 4 et centrée. Utiliser la fonction *ma()*. Tracer la série et la tendance sur un même graphique (fonctions *plot()* et *lines()*).



3. Calcul de la série sans tendance.

En supprimant la tendance précédemment calculée de la série chronologique, on obtient une nouvelle série chronologique qui expose clairement la saisonnalité. En faisant la différence de la série originelle moins la tendance, calculer la série sans tendance, puis la tracer.



4. Calcul de la saisonnalité

A partir des séries chronologiques sans tendances, nous calculons la saisonnalité moyenne. Nous additionnons les variations saisonnières et les divisons par la période de variation saisonnière. Techniquement parlant, pour faire la moyenne des séries chronologiques, nous les introduisons dans une matrice. Ensuite, nous transformons la matrice pour que chaque colonne contienne des éléments de la même période (même jour, même mois, même trimestre, etc...). Enfin, nous calculons la moyenne de chaque colonne.

Ici, nous avons une saisonnalité trimestrielle : nous utilisons une matrice de 4 lignes. La saisonnalité moyenne est répétée 16 fois pour créer le graphique à comparer plus tard.

```
m_beer = t(matrix(data = detrended_beer, nrow = 4))
seasonal_beer = colMeans(m_beer, na.rm = T)
plot(as.ts(rep(seasonal_beer, 16)))
```

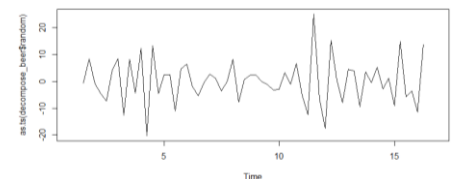
5. Calcul du bruit aléatoire restant

La formule additive est la suivante :

"Série chronologique = Saisonnier + Tendance + Aléatoire" $\Leftrightarrow$ "Aléatoire = Série chronologique - Saisonnier - Tendance". Calculer l'erreur et la tracer.

6. Utilisation de la fonction *decompose()*

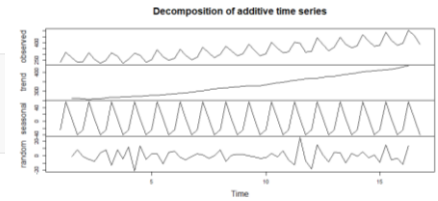
Pour faciliter la vie, certains paquets R fournissent une décomposition avec une seule ligne de code. Comme prévu, notre décomposition étape par étape



donne les mêmes résultats que la méthode `decompose()`. La seule exigence : la saisonnalité est trimestrielle (fréquence = 4). Utiliser la fonction `decompose()` pour retrouver les résultats précédents

```
ts_beer = ts(timeserie_beer, frequency = 4)
decompose_beer = decompose(ts_beer, "additive")
```

Tracer les graphiques de la tendance, de la saisonnalité et du bruit séparément, puis dans une même fenêtre



3. Exercice. Décomposition manuelle de séries : modèle multiplicatif.

Les chiffres mensuels sur les passagers des compagnies aériennes sont un bon exemple de série chronologique multiplicative. Plus il y a de passagers, plus on observe de saisonnalité.

1. Charger les données et les tracer.

```
library(fpp)
library(forecast)
data(AirPassengers)
timeserie_air = AirPassengers
plot(as.ts(timeserie_air))
```

2. Détection de la tendance

La saisonnalité du nombre de passagers des compagnies aériennes semble annuelle. Toutefois, elle est enregistrée mensuellement, de sorte que nous choisissons une fenêtre moyenne mobile centrée de 12. Utiliser la fonction `ma()`. Tracer la série et la tendance sur un même graphique (fonctions `plot.ts()` et `lines()`).

3. Calcul de la série sans tendance.

En supprimant la tendance précédemment calculée de la série chronologique, on obtient une nouvelle série chronologique qui expose clairement la saisonnalité. En faisant la division de la série originelle par la tendance, calculer la série sans tendance, puis la tracer.

4. Calcul de la saisonnalité

A partir des séries chronologiques sans tendances, nous calculons la saisonnalité moyenne. Nous additionnons les variations saisonnières et les divisons par la période de variation saisonnière. Techniquement parlant, pour faire la moyenne des séries chronologiques, nous les introduisons dans une matrice. Ensuite, nous transformons la matrice pour que chaque colonne contienne des éléments de la même période (même jour, même mois, même trimestre, etc...). Enfin, nous calculons la moyenne de chaque colonne.

Ici, nous avons une saisonnalité mensuelle : nous utilisons une matrice de 12 lignes. La saisonnalité moyenne est répétée 12 fois pour créer le graphique que nous comparerons plus tard

```
m_air = t(matrix(data = detrend_air, nrow = 12))
seasonal_air = colMeans(m_air, na.rm = T)
plot(as.ts(rep(seasonal_air, 12)))
```

5. Calculer du bruit aléatoire restant

La formule multiplicative est :

"Série chronologique = Saisonnier * Tendance * Aléatoire" $\Leftrightarrow$ "Aléatoire = Série chronologique / (Tendance * Saisonnier)". Calculer l'erreur et la tracer.

6. Utilisation de la fonction *decompose()*

Pour faciliter la vie, certains paquets R fournissent une décomposition avec une seule ligne de code. Comme prévu, notre décomposition étape par étape donne les mêmes résultats que la méthode *decompose()*. La seule exigence : la saisonnalité est mensuelle (fréquence = 12). Utiliser la fonction *decompose()* pour retrouver les résultats précédents

```
ts_air = ts(timeserie_air, frequency = 12)
decompose_air = decompose(ts_air, "multiplicative")
```

Tracer les graphiques de la tendance, de la saisonnalité et du bruit séparément, puis dans une même fenêtre

4. Exercice. Moyenne mobile et décomposition.

Dans cet exercice, nous travaillerons avec le jeu de données *elecequip* de la librairie *fpp*. Il s'agit de données de demande de production européenne d'équipements électriques (ordinateurs, produits électroniques, produits optiques), source Eurostat.

```
library(fpp)
data = elecequip
```

1. Décrire les données (sujet, type de données, nombre d'entrées, pas de temps), les tracer (fonction *plot.ts()*)

2. Lissage par moyenne mobile

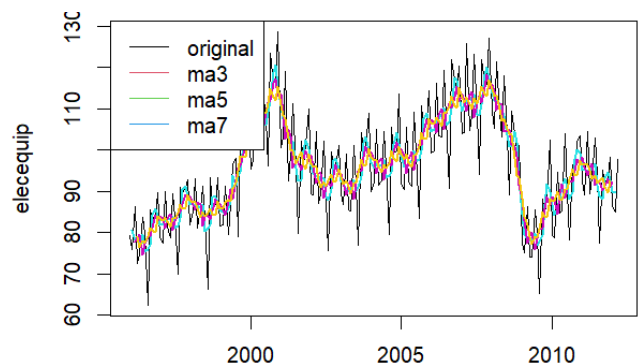
Une première méthode de lissage, dite non-paramétrique, est l'utilisation d'une moyenne mobile (MA=Moving Average). Le but est d'approcher la tendance en atténuant les fluctuations irrégulières tout en éliminant la composante saisonnière.

21. Utilisez la fonction *ma()* du package *forecast* pour calculer la moyenne mobile centrée d'ordre $k=3$ ou 5 ou 7.

22. Visualisez le résultat grâce à un graphique. Jouez avec l'ordre de la moyenne mobile pour en observer l'effet.

23. Pourquoi les données aux extrémités sont égales à des NAs ?

24. Utilisez la fonction *rollmean()* du package *zoo* pour calculer la moyenne mobile centrée d'ordre $k=3$ ou 5 ou 7.



3. Décomposition via un modèle additif

31. Calculer la décomposition du signal *elecequip* selon un modèle additif. Tracer le graphique qui résume la décomposition par ce modèle additif

32. Calculer la décomposition du signal *elecequip* selon un modèle multiplicatif. Tracer le graphique qui résume la décomposition par ce modèle multiplicatif.

33. Quelles composantes des décompositions varient selon un modèle ou un autre ?

34. Est-ce que les composantes de tendance et saisonnière expliquent bien le signal ?

TP03- SERIES CHRONOLOGIQUES : PREDICTION

Objectif: lissage exponentiel simple, double ou de type Holt-Winters, études de résidus issus de la modélisation

1. AIDE SOUS R

Découpage de série temporelle

```
serie_decoupee<-window(serie,start = une_date_1 ,end = une_date_2)
```

Modélisation par lissage exponentiel

```
#Lissage exponentiel simple :
serie_modelisee_LS = HoltWinters(serie,alpha=α,beta=FALSE,gamma=FALSE),
# un lissage exponentiel double paramètre 1-α0 :
serie_modelisee_LD = HoltWinters(serie,alpha=α, beta=NULL, gamma=FALSE) #avec α =1-(α0)^2,
β=(1-α0)/(1+α0)
#un lissage de Holt-Winters sans composante saisonnière :
serie_modelisee_LHW_1 = HoltWinters(serie,alpha=α,beta=β,gamma=FALSE),
#un lissage Holt-Winters additif:
serie_modelisee_LHW_2 = HoltWinters(serie,alpha=α,beta=β,gamma=γ,seasonal="add"),
# un lissage Holt-Winters multiplicatif :
serie_modelisee_LHW_1 = HoltWinters(serie,alpha=α,beta=β,gamma=γ,seasonal="mul")
#Remarque : lorsqu'aucune valeur n'est précisée pour les constantes de lissage, un
#algorithme interne à la procédure HoltWinters se charge d'estimer la meilleur constante
#possible à partir de #la série des observations
```

Prévisions

```
prevision_10 = predict(serie_modelisee,n.ahead=10) # prévisions à l'horizon h = 10
#on peut aussi utiliser library(forecast)
install.packages("forecast",repos="http://R-Forge.R-project.org")
library(forecast)
prevision_10 <- forecast.HoltWinters(serie, h=14)
plot.forecast(prevision_10)

# Tracer la série, la serie modelisée et la prédiction sur le même graphique :
plot(serie_modelisee, prevision_10 )

par(mfrow=c(n,p)) #afficher n*p graphique sur la même page
```

Résidus

```
#HoltWinters() ne sauve pas les résidus, mais on peut les récupérer avec
mes_residus = serie_originale - model$fitted[,1]
mes_residus = model$x- model$fitted[,1]
plot.ts(mes_residus,main="Residus du modele",ylab="Nombres de passagers (x1000)")
abline(h=0,col="red")
qqnorm(mes_residus) #create Q-Q plot for residuals
qqline(mes_residus,col="red") #add a straight diagonal line to the plot
hist(mes_residus, main = "Histogramme des résidus et densité associée",freq= FALSE)
lines(density(mes_residus), col = "red", lwd = 2)
```

1. Exercice. Prédiction par lissage de valeur boursières.

On considère le cours de clôture quotidiens du CAC40. L'ensemble de données EuStockMarkets contient les cours de clôture quotidiens (à l'exception des week-ends/jours fériés) de quatre bourses européennes : le DAX (Allemagne), le SMI (Suisse), le CAC (France) et le FTSE (Royaume-Uni). Le but est d'essayer de prédire par lissage exponentiel les valeurs de clôture de 1998 en utilisant les valeurs de clôture de 1991 à 1997.

1. Charger les données de CAC40, les plotter, les décrire :

De quoi s'agit-il :

Quel est le type d'objet manipulé :

Début :

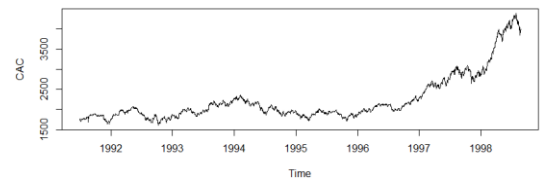
Fin :

Fréquence :

A quoi correspond 260 ?

```
library(datasets)
library(tseries)

data(EuStockMarkets)
View(EuStockMarkets)
CAC=EuStockMarkets[,3]
```



2. Découper le jeu de données en deux sous-séries :

- La série x_{train} : du début jusqu'à fin 1997 qui sera utilisée comme base pour la prédiction

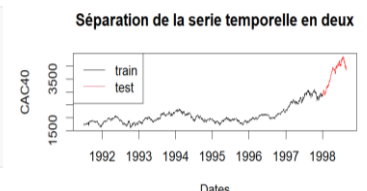
```
x_train <- window(CAC,end=c(1997,260)) # données jusqu'au fin 1997
l_train <- length(x_train)
```

- La série x_{test} : les données depuis 1998 qui sera utilisée comme base pour validation de la prédiction

Donner les longueurs des deux sous séries x_{train} et x_{test} ?

On trace alors les deux séries dans un même graphique :

```
seqplot.ts(x_train, x_test, colx = "blue", coly = "red",
           ylab = "CAC40", xlab = "Dates",
           main = "Séparation de la serie temporelle en deux",)
legend("topleft", c("train", "test"), col=1:2, lty=1:1)
```



La fonction `HoltWinters()` permet d'effectuer les lissages exponentiels

3. Lissage exponentiel simple

La fonction `HoltWinters()` avec les options `beta=FALSE` et `gamma=FALSE` permet de modéliser via des un lissage exponentiel simple

```
x_train_modeled_LS =HoltWinters(x_train, alpha=NULL, beta=FALSE,gamma=FALSE)
```

Combien de parametres sont necessaires pour construire un lissage simple ?

Quelles sont les valeurs de alpha, beta et gamma ?

```
x_train_modeled_LS$alpha #[1] 0.999
```

Qu'est-ce que la ligne ci-dessous permet de calculer ?

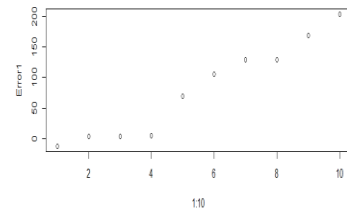
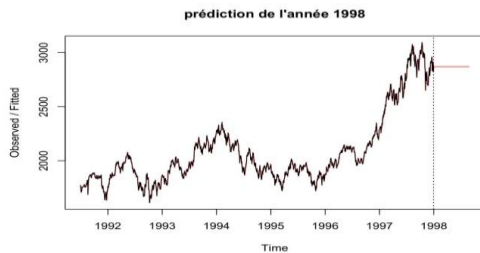
```
x_train_modeled_LS$SSE    #[1] 1238819
```

Faisons des predictions, puis visualison l'erreur entre la serie observée et la serie modelisée.

```
x_test_mod <- predict(x_train_modeled_LS,n.ahead=length(x_test))
plot(x_train_modeled_LS,x_test_modLS,
     main = "prédiction de l'année 1998 via LES")
lines(x_test,col= "blue")
legend("topleft", c("train","test","prediction"),col=1:3,lty=1:1)
```

Pour les erreurs

```
Error1 <- x_test[1:10]-x_test_mod[1:10]
plot(1:10>Error1)
```



4. Lissage double

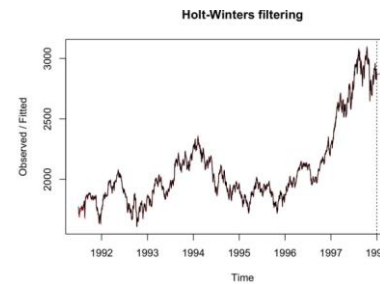
41. Prédire par lissage exponentiel double (2 paramètres) via la fonction *HoltWinters()* avec les options *beta=NULL* et *gamma=FALSE*.

Ploter la série en noire, le lissage exponentiel en rouge.

Donner les coefficient α et β donnés par l'algorithme, la somme finale des erreurs au carré obtenues lors de l'optimisation.

42. Faire une prédiction pour l'année 1998 avec la fonction *predict()* et l'option *n.ahead=longueur de 1998*

43. Ploter la série en noire, le lissage exponentiel en rouge, et la prédiction en bleu.



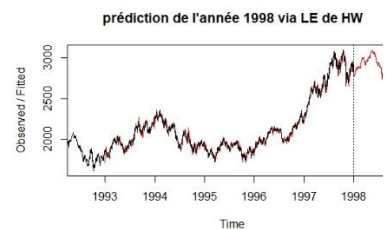
5. Lissage de HoltWinter

51. Prédire par lissage de HoltWinter (3 paramètres) via la fonction *HoltWinters()* avec les options *beta=NULL* et *gamma=NULL*.

Donner les coefficient α , β et γ donnés par l'algorithme, la somme finale des erreurs au carré obtenues lors de l'optimisation.

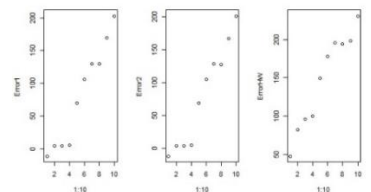
52. Faire une prédiction pour l'année 1998 avec la fonction *predict()* et l'option *n.ahead=longueur de 1998*

53. Ploter la série en noire, le lissage exponentiel en rouge, et la prédiction en bleu.



6. Quelle est la différence entre les trois prédictions : une paraît mieux adaptée, une autre constate et l'autre seulement linéaire.

En comparant les graphes des erreurs ploté aux question 33 et 43 que pouvez-vous dire de plus ?



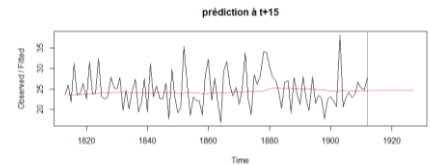
2. Exercice. Pluie à Londres.

1. Charger les données suivantes. Il s'agit des cumuls annuels de pluie à Londres entre 1813 et 1912

```
rain <- scan("http://robjhyndman.com/tsdldata/hurst/precip1.dat", skip=1)
```

2. Transformer le vecteur *rain* en un objet *ts* dont la fréquence est annuelle, et les date vont de 1

3. Faire une prédiction par lissage exponentiel simple à l'horizon $t+15$. Pour cela, on utilise la fonction *HoltWinters()* avec les options *beta=FALSE* et *gamma=FALSE*.

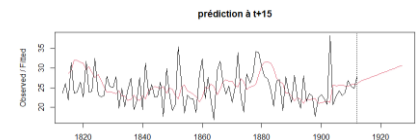


31. Tracer le résultat de ce lissage

32. Donner le coefficient α donné par l'algorithme, la somme finale des erreurs au carré obtenues lors de l'optimisation.

33. Sauvegarder les résidus (c.à.d. la série d'origine moins la série modélisée), les ploter.

4. Faire une prédiction par lissage exponentiel double à l'horizon $t+15$. Pour cela, on utilise la fonction *HoltWinters()* avec les options *beta=NULL* et *gamma=FALSE*.



41. Tracer le résultat de ce lissage

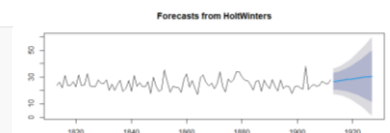
42. Donner les coefficients du modèle exponentiel double donné par l'algorithme, la somme finale des erreurs au carré obtenues lors de l'optimisation.

43. Sauvegarder les résidus (c.à.d. la série d'origine moins les résidus), les ploter.

5. Prédire par de HoltWinter via la fonction *HoltWinters()* avec les options *beta=NULL* et *gamma=NULL*. Que se passe-t-il ? Proposer une explication.

6. Charger la librairie *forecast*. Etablir des prévisions aux horizons 15 et 30 en utilisant la fonction *forecast()*

```
install.packages("forecast", repos="http://R-Forge.R-project.org")
library(forecast)
rs15 <- forecast(rsH, h=15)
```



3. Exercice. Concentration en co2 à Hawaï.

Le fichier de données *co2* contenu dans R contient les concentrations mensuelles en CO2 (en parties par million) à proximité du volcan Mauna Loa, Hawaï, USA, de 1959 à 1997.

1. Description des données

Faire une régression linéaire (données ~ temps)

Tracer les résidus

Les résidus suivent-ils une loi normale ?

2- Analyse de série temporelle

Décomposer la série en un modèle additif.

Visualiser la décomposition

Tracer le graphe de la série ajustée (cad la série initiale sans sa composante saisonnière)

3- Prédiction

Utiliser la fonction *ets()* du package *forecast* pour réaliser un lissage exponentiel. Stocker ce model dans une variable *CO2forecasts*.

Visualiser la prédiction et l'original dans une même fenêtre

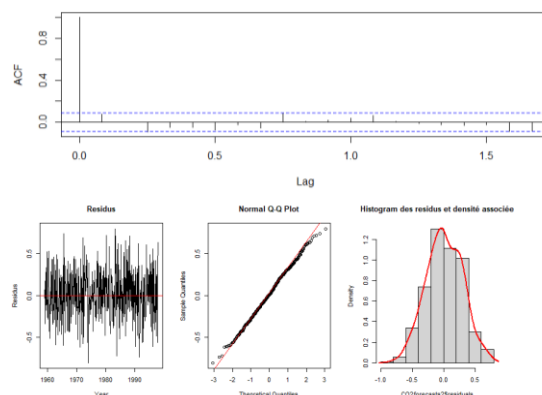
Faire une prédiction à 44 mois

4- Autres

Tracer le graphe des autocorrélations, fonction *acf()*

Faire un test de Box-Ljung (énoncer les hypothèses associées au test), fonction *Box.test()*.

Pour le modèle, tracer les résidus en fonction du temps, le graphique Q-Qplot des résidus, puis les graphes des histogrammes avec la densité associé (voir figure ci-contre). Les résidus vous semblent ils normaux ?



TP04- TEST SUR LES SERIES CHRONOLOGIQUESObjectif :

Test de détection de la saisonnalité

Analyse de la variance et test de Fisher

Test de Buys-Ballo

Test de detection de la tendance

Le test de Mann-Kendall sert à déterminer avec un test non paramétrique si une tendance est identifiable dans une série temporelle qui comprend éventuellement une composante saisonnière. L'hypothèse nulle H_0 de ces tests est qu'il n'y a pas de tendance. Les trois hypothèses alternatives de tendance négative, non nulle ou positive peuvent être choisies. Les tests de Mann-Kendall s'appuient sur le calcul du tau de Kendall mesurant l'association entre deux échantillons et lui-même basé sur les rangs à l'intérieur des échantillons.

Le test de Cochran-Armitage permet de tester si des proportions, éventuellement calculées à partir d'un tableau de contingence, peuvent être considérées comme variant linéairement en fonction d'une variable ordinale ou continue.

Test de stationarité/Non-stationnarite

Les tests de stationnarité permettent de vérifier si une série est stationnaire ou non. Il y a deux approches différentes : les tests pour lesquels l'hypothèse nulle H_0 est que la série est stationnaire (test KPSS de Leybourne et McCabe), et ceux pour lesquels l'hypothèse nulle est au contraire que la série n'est pas stationnaire (test de Dickey-Fuller, test augmenté de Dickey-Fuller, test de Phillips-Perron, test DF-GLS).

Tests de bruit blanc

Test des autorcorralltion

Test de normalité

4. TP2>Exercice A PREPARER.

5- A faire sur CO2 Manau loa

2. Quel modèle de lissage exponentiel simple vous semble le mieux approprié ?

3. Afin de valider ce modèle, tester la prédiction des données de 1990 à 1997 en utilisant celles de 1959 à 1989.

31. Découpé le jeu de donner Data1=données de 1959 à 1989 et DataVal=données de 1990 à 1997

32. Etablir un modèle LES basé sur les données de 1959 à 1989, et sa prévision associée des années 1990 à 1997

33. Calculer l'erreur quadratique moyenne (RMSE) entre les données prédites et DataVal

4. Si le modèle prévisionnel établi vous semble correct vous semble graphiquement correct, utilisez cette méthode pour prédire les concentrations en CO2 de 1997 à 2007.

5. Tester d'autres méthodes de lissage exponentiel double et Holt-Winter

5. TP2>Exercice A PREPARER.

BEAUCOUP d'exo simple sur
http://staff.univ-batna2.dz/sites/default/files/merahi-fateh/files/methode_chronologique_et_prevision-chapitre_12.pdf